AAST College of Engineering
Department of Computer Engineering

# Complexity Analysis

Noha Seddik Abdelsalam

# Introduction to Algorithm analysis

# Why we need to understand program complexity

- Consider the following ‘Pseudo code’ example :

```
Def find_sum(given_array):
  total = 0
  for each i in given_array:
        total = total + i
Return total
```

**How much time does it take to run this function ?**

We don't know

It Depends on:

- Programming language you use
- Computer specs (how fast computer is)
- Do you run other programs on the computer

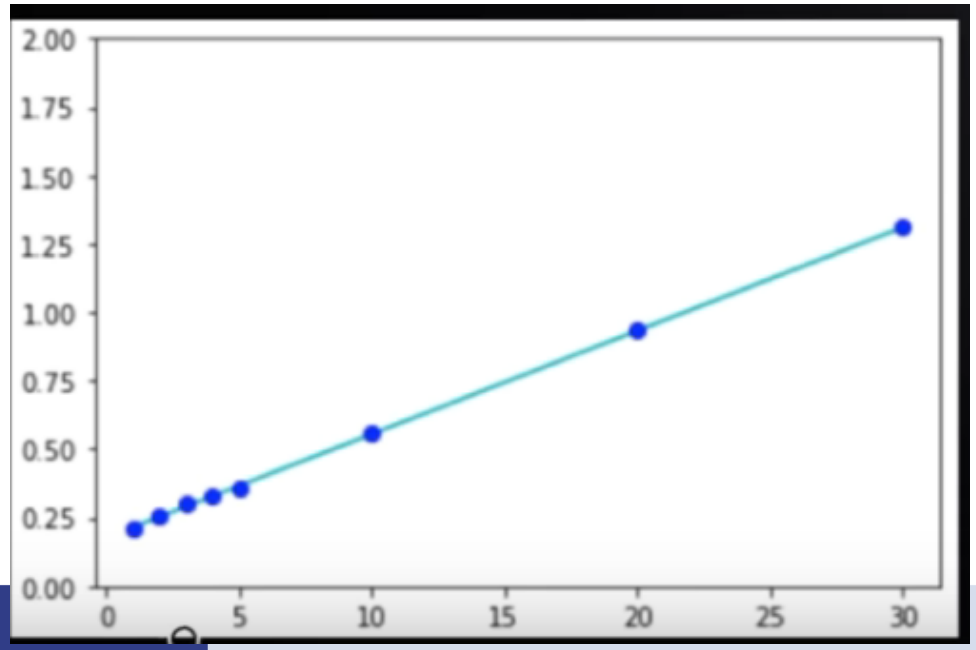
**How much time does it take to run this function ?**

Rephrase …

Big O Notation

**How does the runtime of this function grow WRT the input**

Time … in micro second




N … length of the array

# What is Time complexity

- To know the time needed to complete the function, **we use Big O notation as a way of showing how the runtime of the function increases when the input size increases**.

- To give a roughly idea how **your function scales when the input to that function becomes larger** & larger

- **Big O** notation measures **worst-case** (Upper bound).

# How to calculate Big O notation

1. Different steps get added
2. Drop constants
3. Different inputs implies different variables
4. Find the fastest growing term
5. Take out the coefficients

- if $T = 5000$ $= O(1)$
- if $T = aN + b$ $= O(N)$
- if $T = cN^2 + dN + e$ $= O(N^2)$
- if $T = xN^3 + yN^2 + zN + w$ $= O(N^3)$

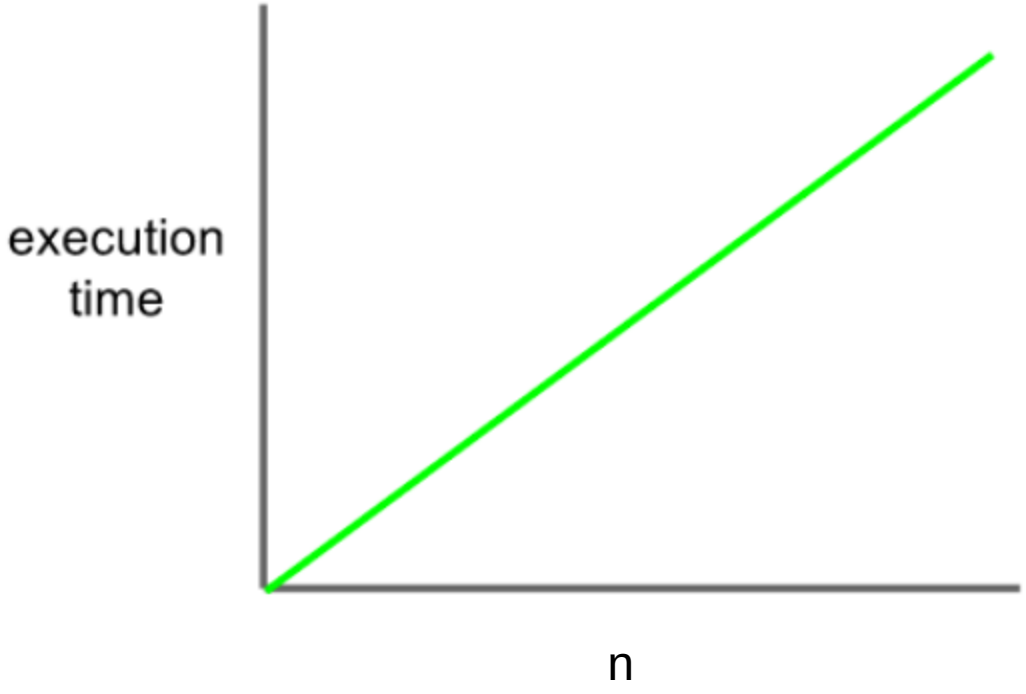
# Time / computational complexity

- Constant time $O(1)$
- Linear time $O(n)$
- Logarithmic time $O(\log n)$
- Quasilinear time $O(n \log n)$
- Quadratic time $O(n^2)$
- Cubic time $O(n^3)$
- Exponential time $O(2^n)$
- Factorial time $O(n!)$

# Linear time O(n)

for (int i = 0; i < n; **i = i + 1**) // i++

Example: Linear search

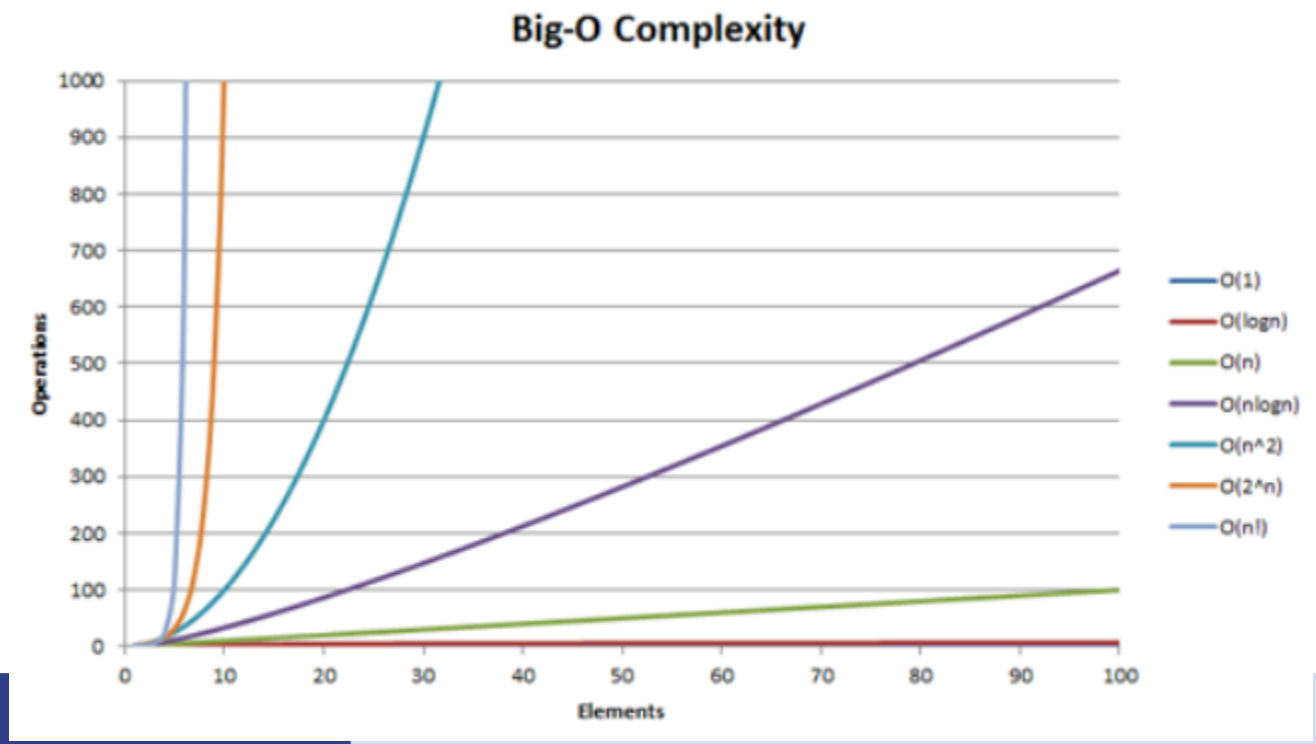


# Quadratic time $O(n^2)$

- for (int i = 0; i < n; **i = i + 1**) {

  for (int j = 0; j < n; **j = j + 1**)

- Example: bubble sort



# Cubic time $O(n^3)$

- for (int i = 0; i < n; **i = i + 1**) {

  for (int j = 0; j < n; **j = j + 1**) {

  for (int z = 0; z < n; **z = z + 1**)
- Example: multiplication of two *n***n* matrices

# Exponential time $O(2^n)$

- function fib(n)

```
{
   if(n <= 1)
     { return n  }
  else
     return fib(n-2) + fib(n – 1)
 }
```

Example:  traversing all nodes in binary tree

# Factorial time O(n!)

- function fact(n) {

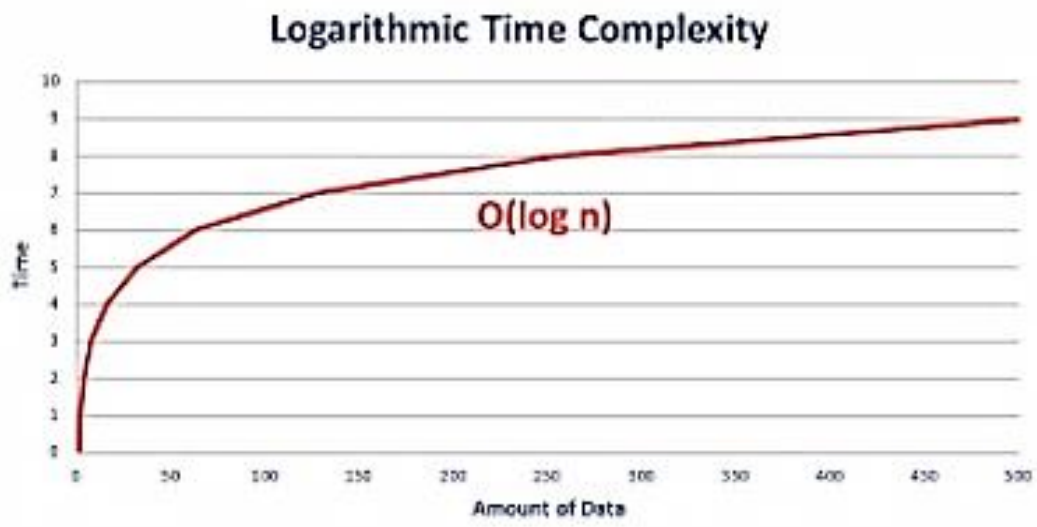
```
    for(i=0; i<n;  i++)
      {
            fact(n-1);
      }
}
```

Example: generating permutations

# Logarithmic time $O(\log_2 n)$

- for (int i = 0; i < n; **i = i * 2**)
- Example: Binary search

| Data | Time |
|---|---|
| 1 | 0 |
| 2 | 1 |
| 4 | 2 |
| 8 | 3 |
| 16 | 4 |
| 32 | 5 |
| 64 | 6 |
| 128 | 7 |
| 256 | 8 |
| 512 | 9 |

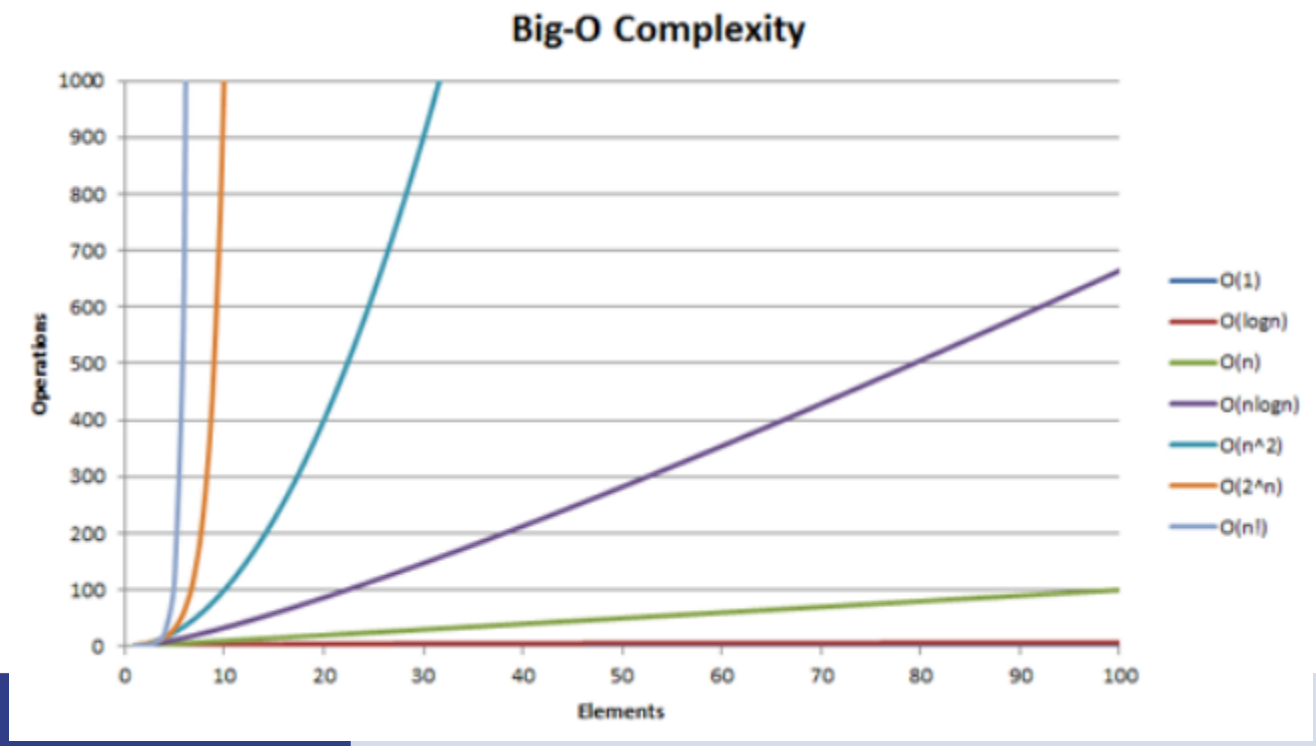


# Quasilinear time $O(n \log_2 n)$

- for (int i = 0; i < n; **i = i + 1**) {

  for (int j = n; j > 0; **j = j / 2**)

- Example: merge sort



# Constant time O(1)

- If running time does not depend on the size of the input (n)
- O(***any constant value***), this is equivalent to being O(1). Even if there are loops, it doesn't depend on the size of n

```
int index = 5;
int item = list[index];
if (condition true) then
   perform some operation that runs in constant time
else
   perform some other operation that runs in constant time
for i = 1 to 100
   for j = 1 to 200
      perform some operation that runs in constant time
```

telerik

# Time Complexity and Speed

| Complexity | 10 | 20 | 50 | 100 | 1 000 | 10 000 | 100 000 |
|---|---|---|---|---|---|---|---|
| O(1) | < 1 s | < 1 s | < 1 s | < 1 s | < 1 s | < 1 s | < 1 s |
| O(log(n)) | < 1 s | < 1 s | < 1 s | < 1 s | < 1 s | < 1 s | < 1 s |
| O(n) | < 1 s | < 1 s | < 1 s | < 1 s | < 1 s | < 1 s | < 1 s |
| O(n*log(n)) | < 1 s | < 1 s | < 1 s | < 1 s | < 1 s | < 1 s | < 1 s |
| $O(n^2)$ | < 1 s | < 1 s | < 1 s | < 1 s | < 1 s | 2 s | 3-4 min |
| $O(n^3)$ | < 1 s | < 1 s | < 1 s | < 1 s | 20 s | 5 hours | 231 days |
| $O(2^n)$ | < 1 s | < 1 s | 260 days | hangs | hangs | hangs | hangs |
| O(n!) | < 1 s | hangs | hangs | hangs | hangs | hangs | hangs |
| $O(n^n)$ | 3-4 min | hangs | hangs | hangs | hangs | hangs | hangs |

Source: https://www.slideshare.net/introprogramming/19-algorithms-andcomplexity

telerik

# Complexity Examples

```
int FindMaxElement(int[] array)
{
    int max = array[0];
    for (int i=0; i<array.length; i++)
    {
        if (array[i] > max)
        {
            max = array[i];
        }
    }
    return max;
}
```

- Runs in O(n) where n is the size of the array
- The number of elementary steps is ~ n

Source: https://www.slideshare.net/introprogramming/19-algorithms-andcomplexity

telerik

# Complexity Examples (2)

```
long FindInversions(int[] array)
{
    long inversions = 0;
    for (int i=0; i<array.Length; i++)
        for (int j = i+1; j<array.Length; i++)
            if (array[i] > array[j])
                inversions++;
    return inversions;
}
```

- Runs in $O(n^2)$ where n is the size of the array
- The number of elementary steps is ~ n*(n+1) / 2

Source: https://www.slideshare.net/introprogramming/19-algorithms-andcomplexity

telerik

# Complexity Examples (3)

```
decimal Sum3(int n)
{
    decimal sum = 0;
    for (int a=0; a<n; a++)
        for (int b=0; b<n; b++)
            for (int c=0; c<n; c++)
                sum += a*b*c;
    return sum;
}
```

- Runs in cubic time $O(n^3)$
- The number of elementary steps is $\sim n^3$

Source: https://www.slideshare.net/introprogramming/19-algorithms-andcomplexity

telerik

# Complexity Examples (4)

```
long SumMN(int n, int m)
{
    long sum = 0;
    for (int x=0; x<n; x++)
        for (int y=0; y<m; y++)
            sum += x*y;
    return sum;
}
```

- Runs in quadratic time O(n*m)
- The number of elementary steps is ~ n*m

Source: https://www.slideshare.net/introprogramming/19-algorithms-andcomplexity

# OTHER EXAMPLES

# Example

```
Def func1(given_array):
  total = 0
Return total
```

= O(1)

```
Def find_sum(given_array):
  total = 0
  for each i in given_array:
      total = total + i
Return total
```

=O(N)

# Example

```
Def find_sum(given_arr2d):
  total = 0
  for each row in given_arr2d:
    for each i in row:
        total = total + i
Return total
```

$=O(N^2)$

# Example

```
Def sum_product(given_arr):
 total = 0
 product = 1
 for each i in given_arr:
        total = total + I
 for each i in given_arr:
        product = product x I
 print sum, product
Return
```

$T = O(1) + O(1) + N \times O(1) + N \times O(1) + O(1)$

$= c_1 + 2 \times N$

$= O(N)$

# Example

```
Def intersection(arrA, arrB):
 count = 0
for each i in arrA:
    for each j in arrB:
        if i = = j
         count = count + I
Return count
```

$T = O(1) + O(N) \times O(M) \times O(1) + O(1)$

$= c_1 + O(N) \times O(M) \times O(1)$

$= O(N \times M)$

Where N & M are the size of arrA & arrB respectively

# Thank You